Maren

CONFIDENTIAL

Maren Finance
Progress & Evidence

What has been built so far, deployed where, proven how, and the exact commands and transaction hashes that let you check every claim without trusting us.

DOCUMENT	DATE	VERSION	PREPARED FOR
MRN-100	17 August 2026	1.3	Mezo Ecosystem

DOCUMENT	DATE	CLASSIFICATION	VERSION
MRN-100	17 August 2026	Confidential	1.3

Maren Finance — Progress & Evidence

What has been built so far, deployed where, proven how, and the exact commands and transaction hashes that let you check every claim without trusting us.

CONTENTS

1 What Maren is, in one paragraph 2

2 What was built 2

2.1 Preflight before signature 2

2.2 Nothing rendered that cannot be proven 3

3 Deployed contracts, Mezo matsnet (31611) 3

4 Exercised on-chain, not just deployed 4

4.1 The pooled trove is live inside Mezo's own TroveManager 4

4.2 A real user borrowed through the app 5

4.3 Names work, and cannot be front-run 5

4.4 Real user flows, on the real chain 5

4.5 One honest failure, kept on the record 5

5 Test evidence 6

6 The product, as it runs today 6

7 Findings from reading the chain 8

8 Where the numbers a user sees come from 9

9 Not done, stated plainly 9

10 Reproduce every number 10

1

What Maren is, in one paragraph

Maren is a consumer borrowing and payments app for Mezo, on web and iOS. Users deposit Bitcoin, borrow MUSD against it from as little as 0.001 BTC, and spend it: pay a `@name`, share a payment link, request money. It is built for Mezo and deployed nowhere else. Everything below is what exists today, with the evidence.

2

What was built

The codebase is a single monorepo with five parts, sharing one SDK so that every number a user sees is computed in exactly one place.

COMPONENT	WHAT IT IS
Contracts	Five Solidity contracts on matsnet: pooled vault, payments, name registry, ERC-4626 yield vault, ERC-2771 relay hub
SDK	One TypeScript package holding every protocol constant, address, and piece of position math. iOS and web call the same functions
Web app	Portfolio, Vault, Pay, Earn, Activity. Live contract reads; the write path is wired and exercised on testnet
iOS app	Expo, 18 screens, verified in the simulator including the degraded states
Relayer	Gasless transaction service: verifies, simulates, submits, settles gas. Built and fully tested, not yet hosted

Two design decisions shape everything else.

2.1

Preflight before signature

The interface evaluates every rule the deployed contract enforces, in the contract's own order, before a control is enabled. An action Maren offers is an action that clears; a refusal states the same reason the revert would have given, with live numbers. Nobody pays gas to discover a "no".

2.2

Nothing rendered that cannot be proven

Activity is read from contract logs; every row carries a real transaction hash linked to the explorer. There is no sample data anywhere in the product, and the empty states say precisely what was scanned before concluding there was nothing.

3

Deployed contracts, Mezo matsnet (31611)

Five contracts, all source-verified on Blockscout.

CONTRACT	ADDRESS
MarenVault	`0x46252B00AB59384518d9dB6D1f210ffF1221645b`
MarenPay	`0xe475Ff919D54E055809f516E3bed8a5c7a101836`
MarenRelayHub	`0x22cC1Adfb0C91D9359746dE3F7E8cB7F2B8eA98C`
MarenRegistry	`0x9e123Cdd0CD524812CF9F552d3FAB13d8143b77E`
MarenEarn	`0xE3B02b215085B99Ce61E0F5ea9763Ac637aa7826`

Deployment took 11 transactions from the deployer `0x54196397D6f01b48739dB3957257cB33cD341885`, costing 11,688,063 gas in total, about 4 nano-BTC. Every hash below resolves on `explorer.test.mezo.org`. MarenRelayHub is the single trusted forwarder; `MarenPay.trustedForwarder()` confirms it on-chain.

CONTRACT	DEPLOY TRANSACTION
MarenVault	`0x25250742fbd112c08e31611be30a818fbd79bf04a722f672c115dffec901d622`
MarenPay	`0xa91c3ea4ca9b6186f799ace9d5bfde9e8b3c2c689d0d5c0405f280b3835dc807`
MarenRelayHub	`0xc1fdfd7e16a67b6ef7d6cc41cf95795bc17af48ccbca958b6b6ae42a9ea0db74`
MarenRegistry	`0x1a48c318fa00ad914b466f6b5fb665bedf9c1ca77ed8b1ac97e45a0c82f4eae4`
MarenEarn	`0x47d7aea8ee6f27ad87a45129aed94db45ccd60ea3aa8003ef33feb6f794046c3`

4

Exercised on-chain, not just deployed

Deployment proves compilation. What follows proves operation. Every transaction referenced in this section is listed here in full, and each resolves on `explorer.test.mezo.org`.

ACTION	TRANSACTION
Open the pooled trove	`0x13ea27a7b5c9499640c48672028c96e3208aa9924fc5802578cc1044314e93f8`
Register `@sele`, commit	`0x91b7d9515e83792c16f173a695205399263afa061b957cc7652cd2bb915cad04`
Register `@sele`, reveal	`0x048451db7af36cf8a52da611906b256d54cd73f7923c8ad8f4070f88eb666581`
Deposit, scripted	`0xda9a536c732db989a7e6fb56d8676fed7e73f3e30fdc4106d20d37577472602b`
Keeper top-up	`0xb3b90cc57849ac659404802e36b496006e6f42ea57b8b3210f353dd0e101c7f2`
Protocol collateral, 17 Aug	`0x968dd8d9b849655e11cc908f9b8fcd3ef417ae414e5ac8e8a16668e9fd48a8c8`
Keeper sweep, 17 Aug	`0xdc0030b9b16b1fb24a3a329e5b113253ff4b0c1096b1c5712d8a44fe2c4d3cb8`
User deposit, via the app	`0x3b34e664140c594c3b10f4b500e4ea2e582f00002086485b93c972f1695e8dd2`
User deposit, via the app	`0xb9d2f965c3da71ec62e731c8366f8a97c6919cb597974a89675b313f4d70ce63`
User withdrawal, via the app	`0x58798d759af23f9a9bf44d237577209f92f693cca618c68c05052540ea30cbb3`
User borrow, via the app	`0x28bb5ec1d065dc7bd1c1ab4c1997fb4eeb4a75af492bd38fd90442c888427ef2`
Payment request, via the app	`0x059f329fda15d6eb430d514040f7060fe41b354eac478c2c9ad4e5dba490559a`
Deleverage no-op	`0x71adc0a5477cb64c95073360f6a984adb4a9c04f5a8fcc5b73d4249e5456c54d`

4.1

The pooled trove is live inside Mezo's own TroveManager

Opened with `openPooledTrove` at 762,931 gas. It currently holds 0.069 BTC of collateral against 2,001.9 MUSD of debt: an ICR of **220%**, with borrowing enabled. One call verifies it:

BASH

```
cast call 0xE47c80e8c23f6B4A1aE41c34837a0599D5D16bb0 \
  "getTroveColl(address)(uint256)" \
  0x46252B00AB59384518d9dB6D1f210ffF1221645b \
  --rpc-url https://rpc.test.mezo.org
```

4.2

A real user borrowed through the app

On 17 August a user account deposited Bitcoin through the interface and drew **25.7 MUSD** against it, served by the pooled vault in a single 171,053-gas transaction (the user borrow in the log above). The bound is enforced as designed: at the same position, a 40 MUSD borrow reverts with `InsufficientCollateralRatio` at the account's own 180% floor. The interface offers exactly the range the contract accepts, computed before any signature is requested.

4.3

Names work, and cannot be front-run

The `@sele` handle is registered through commit-reveal, the reveal landing at 119,726 gas. `resolve("sele")` returns the deployer address on-chain. Commit-reveal exists so a name cannot be sniped from the mempool.

4.4

Real user flows, on the real chain

Deposits, a withdrawal, keeper collateral top-ups, the borrow above, and an on-chain payment request of 0.50 MUSD, all in the log. Each is a row in the app's Activity screen with its hash linked to the explorer.

4.5

One honest failure, kept on the record

An early `deleverage` call (the last row of the log) reported success while doing nothing: at the protocol's minimum debt there was no repayment room, and the no-op path returned silently. It was found by reading state after the call rather than trusting the receipt, and the fix makes the silent path loud. It is included here because the process that caught it is the same one behind every other number on this page.

5

Test evidence

SUITE	COUNT	RUNS AGAINST
Contract unit and property tests	236	Foundry
Fork tests	8	The real MUSD contracts on matsnet
Relayer	214	Four adversarial review rounds
Web behaviour suite	87	The built app, live matsnet, read-only signer

Three of these deserve a sentence each.

The fork tests run Maren's contracts against the actual deployed MUSD system rather than a mock, so a divergence between MUSD's documentation and its bytecode breaks a test instead of a user.

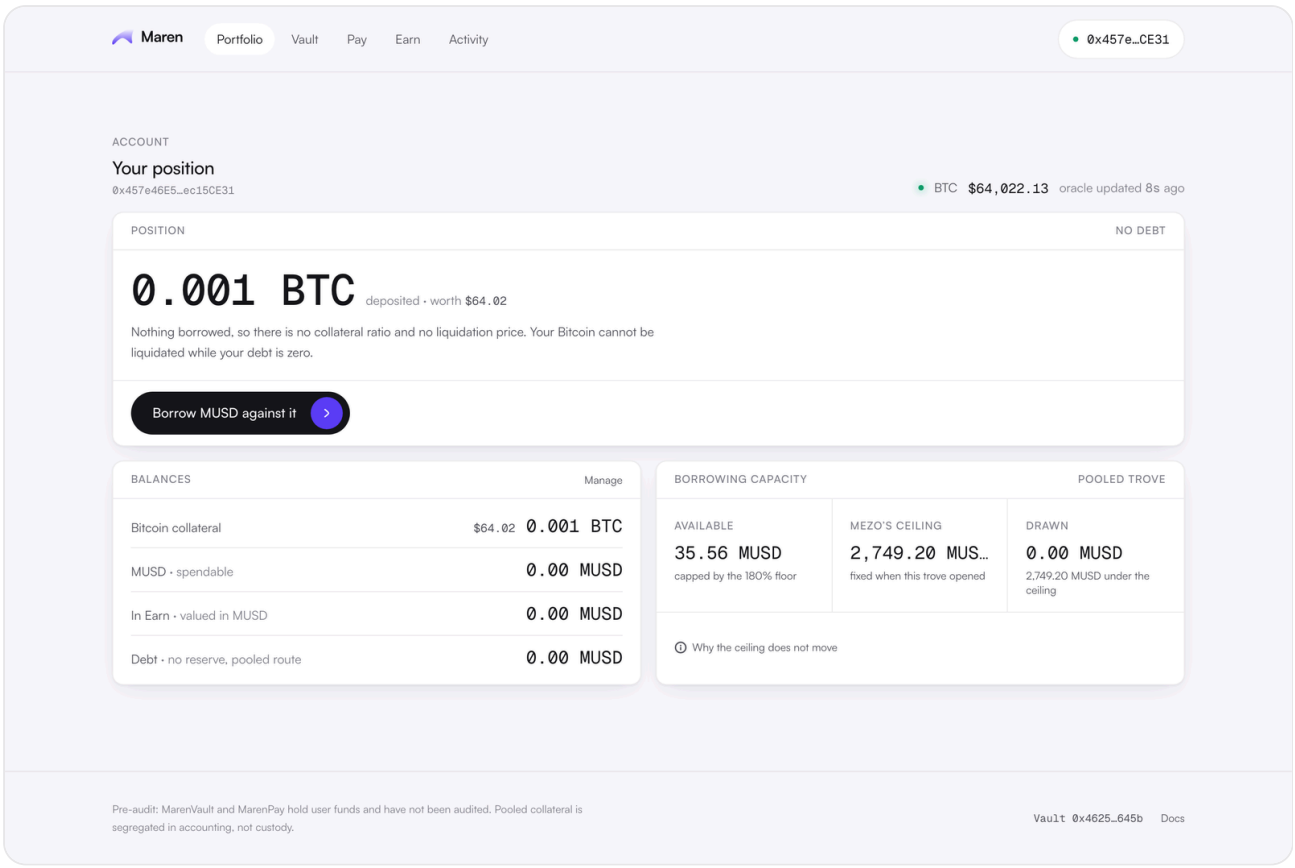
The relayer's four adversarial rounds were structured attacks on our own design. Round two found a budget race allowing 52x overspend; round three found the fix created a denial-of-service path; round four found the CI had never actually exercised the relayer's code. Each finding is fixed and regression-tested.

The behaviour suite drives the real interface against the real chain with a signer that structurally cannot sign, asserts every offered action matches what the deployed contract would accept, and proves no signature is ever requested. The account nonce is read before and after as the final check.

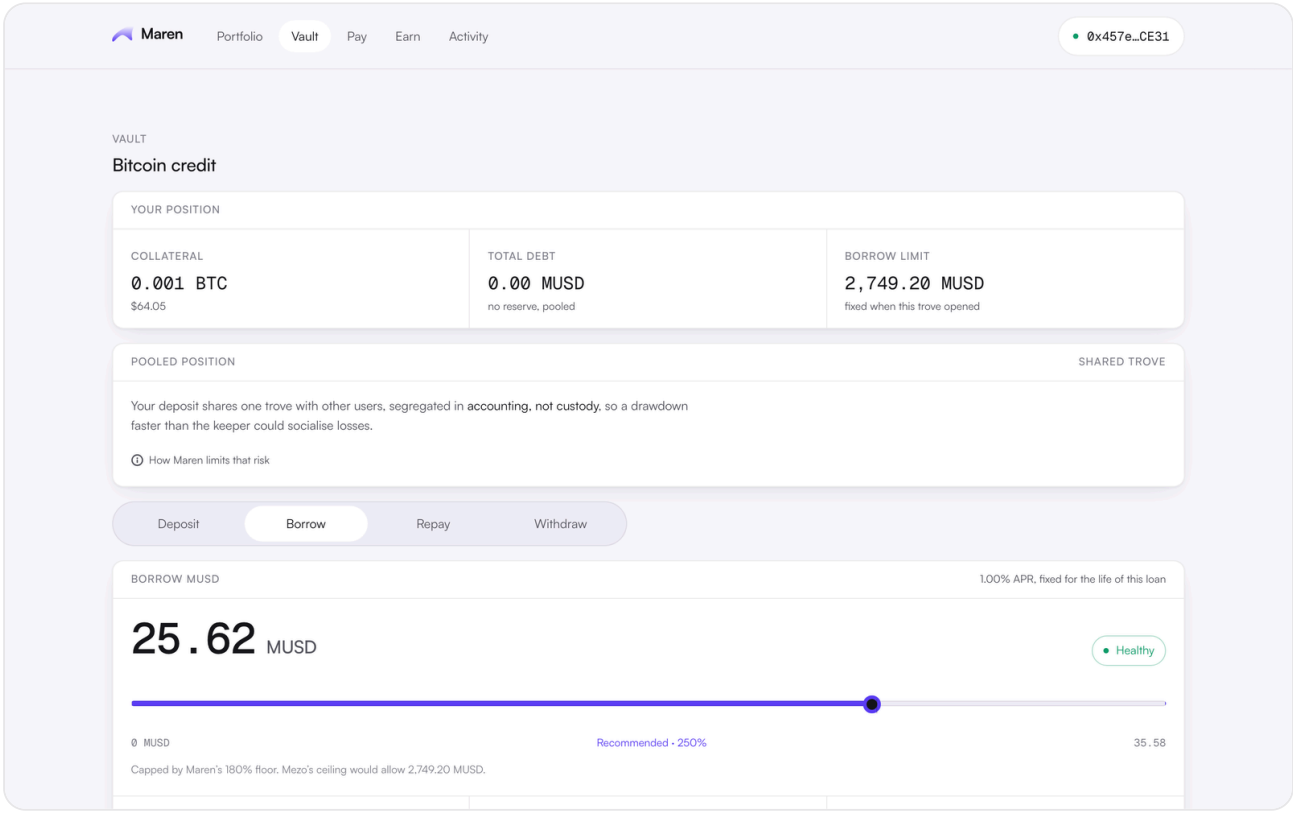
6

The product, as it runs today

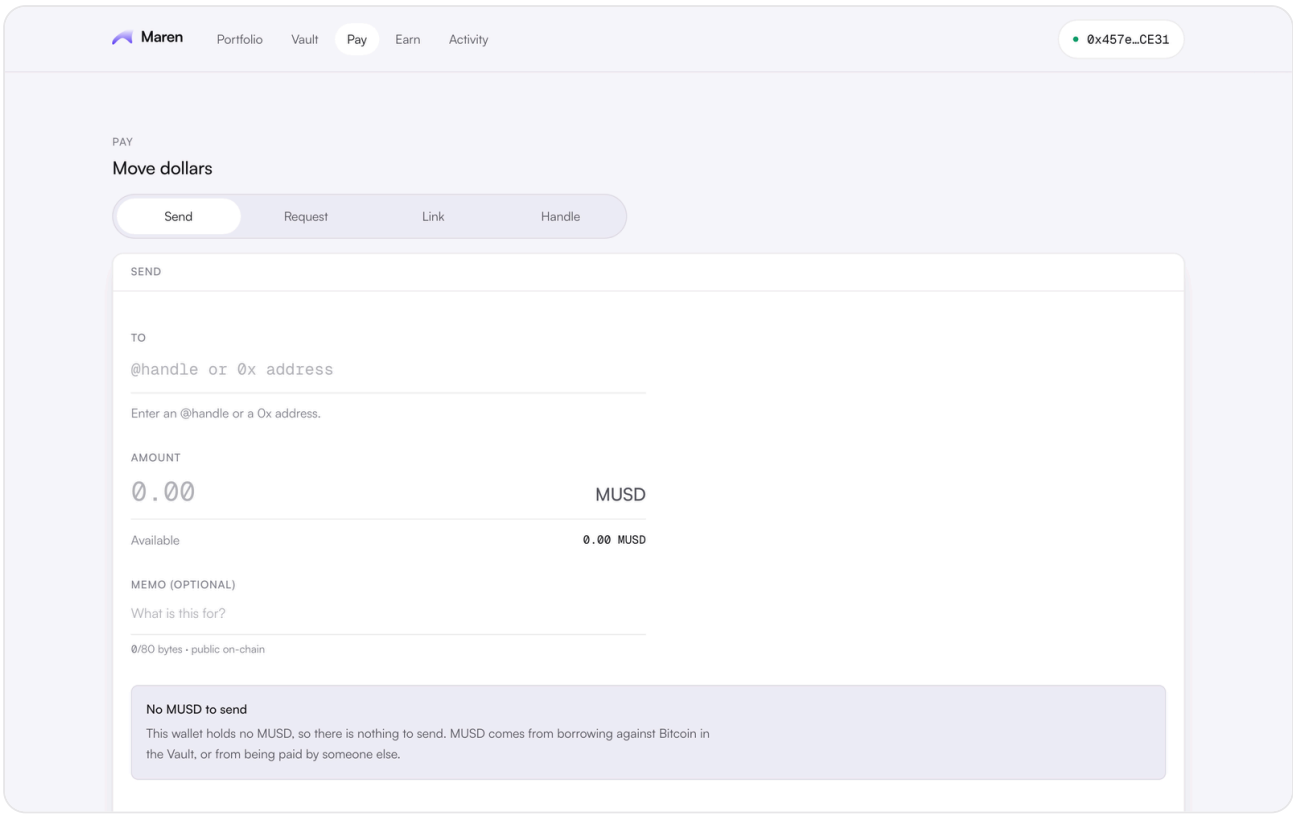
Unretouched captures of the web app in a read-only session against live matsnet, 17 August 2026. Every figure shown is a contract read.



Portfolio. A funded account with no debt: 0.001 BTC deposited, and the interface states plainly that nothing can be liquidated while debt is zero.



Vault. Borrowing live against the pooled trove: the slider is bounded by the same computation the contract enforces, with the recommended 250% marked.



Pay. MUSD moves against @handles or raw addresses, with an on-chain memo. An empty wallet is told where MUSD comes from rather than just refused.

7

Findings from reading the chain

The build began with several days of reading Mezo's deployed bytecode rather than its documentation. That produced findings useful to Mezo regardless of anything else.

1. `viem` **'s Mezo chain definition** ships a default RPC that does not resolve and is missing `multicall3` on mainnet, silently degrading batched reads for every wagmi user on the chain. Fix drafted.
2. **The subgraph documentation** says `network: mezo-mainnet`; Goldsky's actual slug is `mezo`. A manifest copied from the docs is rejected at deploy. Fix drafted.
3. **Three RPC hosts in the developer docs do not resolve.** Fix drafted.
4. **Two places the MUSD docs and deployed bytecode disagree:** the borrower redemption-fee waiver does not appear in `TroveManager`, and the refinance fee computes to roughly 0.02% rather than the documented 0.1%.

5. **ERC-4337 is not viable on Mezo today.** EntryPoint v0.7 is deployed but has processed zero UserOperations per the explorer index, and the RPCs lack `debug_traceCall`, so no compliant bundler can run. Maren uses EIP-2612 permits with ERC-2771 relaying; the chain accepts EIP-7702 transactions as the upgrade path.
6. **The BTC/USD oracle is Mezo's own consensus precompile**, not Chainlink or Pyth, and it reverts above 60 seconds of staleness. Both apps carry an explicit "price unavailable" state and never render a stale price as live.
7. **A sizing result worth passing on:** a pooled vault seeded at the protocol minimum can never serve a borrow, because MUSD's 1,800 minimum net debt pins the seed below its own borrowing floor. We hit this in practice; the pooled trove's move from 153% to 220% ICR is on-chain in two transactions.

8

Where the numbers a user sees come from

Every protocol parameter in the product (the 110% liquidation threshold, the 1,800 MUSD minimum, the 1.00% fixed simple interest, the per-trove borrowing ceiling that adding collateral does not raise) was read from the deployed contracts and recorded in one SDK module with its verification date. Seven documented ways MUSD diverges from the Liquity v1 codebase it descends from live in the same place, because each one breaks a naively ported frontend.

9

Not done, stated plainly

No mainnet deployment yet: it is gated on an independent audit, deliberately. No audit yet. The relayer is built and fully tested but not yet hosted, so today users submit their own transactions and the app says so on every screen. Bank cashout and merchant payments are design work, not code. Nothing in this document overstates that.

10

Reproduce every number

All chain figures re-read in one pass against matsnet and mainnet:

```bash
# matsnet (31611)
cast chain-id --rpc-url https://rpc.test.mezo.org

# mainnet (31612)
cast chain-id --rpc-url https://jsonrpc-mezo.boar.network
```

The addresses and hashes in this document are sufficient to verify independently. The repository additionally carries a verification script that prints each figure beside the command that produced it.

Every chain figure in this document was read live from Mezo RPCs on 17 August 2026, and each one is paired with the command, address, or transaction hash that reproduces it. Counts that drift daily are quoted at the precision at which they hold. Where Mezo's published documentation and the deployed bytecode disagree, this document follows the bytecode and says so.